ASSIGNMENT 13.3

NAME: MOHAMMED FAISAL QURESHI

H.T.NO:2303A53015 BATCH:46

Task 1: Refactoring Data Transformation:

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions

while preserving the output.

Legacy Code

values = [2, 4, 6, 8, 10]

doubled = []

for v in values:

doubled. Append(v * 2)

print(doubled)

Expected Output

[4, 8, 12, 16, 20]

CODE:

Refactored Code (Using List Comprehension):

values = [2, 4, 6, 8, 10]

doubled = [v * 2 for v in values]

print(doubled)

Output:

[4, 8, 12, 16, 20]

Task 2: Improving Text Processing Code Readability

You are working on a log message generator that builds messages word by word.

Task Description

• Analyze the legacy code that constructs a sentence using repeated string concatenation.

• Ask AI to suggest a more efficient and readable approach.

• Refactor the code accordingly while keeping the final output unchanged. Legacy Code

words = ["Refactoring", "with", "AI", "improves", "quality"] message = "" for w in words:

message += w + " "

print(message.strip())

Expected Output

Refactoring with AI improves quality

CODE:

Refactored Code (Using join):

words = ["Refactoring", "with", "AI", "improves", "quality"]

message = " ".join(words)

print(message)

Output:

Refactoring with AI improves quality

Task 3: Safer Access to Configuration Data

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

• Review the legacy code that manually checks for dictionary keys.

• Use AI suggestions to refactor the code using safer dictionary access methods.

• Ensure the behavior remains the same for missing keys.

Legacy Code

config = {"host": "localhost", "port": 8080}

if "timeout" in config:

print(config["timeout"])

else: print ("Default timeout used")

Expected Output

Default timeout used

CODE:

Refactored Code (Using get):

config = {"host": "localhost", "port": 8080}

print (config.get("timeout", "Default timeout used"))

Output:

Default timeout used

Task 4: Refactoring Conditional Logic for Scalability

You are enhancing a utility module that performs different operations based on user input.

Task Description

• Examine the multiple if–elif conditions used to determine operations.

• Ask AI to suggest a cleaner, scalable alternative.

• Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

action = "divide"

x, y = 10, 2

if action == "add":

result = x + y

elif action == "subtract":

result = x - y

elif action == "multiply":

result = x * y

elif action == "divide":

result = x / y

else:

result = None

print(result)

Expected Output : 5.0

CODE:

Refactored Code (Using Dictionary Mapping):

```
action = "divide"
x, y = 10, 2
operations = {
   "add": x + y,
   "subtract": x - y,
   "multiply": x * y,
   "divide": x / y
}
print(operations. Get(action, None))
```

Output:

5.0

Task 5: Simplifying Search Logic in Collections

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

• Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and readable form.

• Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
```

for item in inventory:

if item == "eraser":

found = True

break

print("Item Available" if found else "Item Not Available")

Expected Output

Item Available

CODE:

Refactored Code (Using (in)):

inventory = ["pen", "notebook", "eraser", "marker"]

print("Item Available" if "eraser" in inventory else "Item Not Available")

OUTPUT:

Item Available